Script oral — SamyDessert

Présentation : ~30 min | Démo : ~5 min

> Les phrases sont écrites pour être dites à voix haute, pas lues. Le ⏱ indique le temps cible par slide.

Slide 1 — Titre ⏱ 45s

Bonjour, je m'appelle Samy Ben Hamida, et je vais vous présenter mon projet de fin de formation : SamyDessert, une boutique en ligne de pâtisseries artisanales.

Ce projet couvre l'ensemble du cycle de développement d'une application web : de l'analyse des besoins jusqu'au déploiement en production, en passant par la conception UX, le développement frontend et backend, les tests et la sécurité.

Slide 2 — Sommaire ⏱ 45s

Je vais structurer ma présentation en sept grandes parties. Je commencerai par le contexte et l'analyse des besoins pour expliquer d'où vient le projet et à qui il s'adresse. Ensuite le design et l'identité visuelle, puis l'architecture technique, les fonctionnalités développées, les aspects qualité et sécurité, le déploiement et l'organisation du travail, et je terminerai par un bilan personnel avant la démonstration live.

Slide 3 — Présentation personnelle & projet ⏱ 2min

Je suis en formation Développeur Web et Web Mobile, titre professionnel niveau 5 RNCP, à l'ESRP Auxilia. La formation fonctionne en alternance : une semaine en centre de formation, une semaine en stage. J'effectue mon stage chez Creative Handicap, une association dont l'objectif est de rendre l'art et les métiers du numérique accessibles à tous.

Pour le choix du projet : je prépare régulièrement des desserts que je partage avec mes collègues, et j'ai constaté que les plateformes existantes ont un problème — soit elles vendent des desserts sans expliquer comment ils sont faits, soit elles proposent des recettes trop complexes ou trop coûteuses. Personne ne combine vraiment les deux.

C'est de là qu'est née l'idée : SamyDessert repose sur un concept hybride. Une vitrine pédagogique avec des recettes gratuites pour attirer les visiteurs, et une boutique en ligne pour commander des desserts avec paiement sécurisé. Le contenu gratuit génère de la confiance, la boutique génère du chiffre d'affaires. Je détaillerai les fonctionnalités slide par slide.

Slide 4 — Benchmark & Inspiration ⏱ 1min15

Avant d'écrire la moindre ligne de code, j'ai analysé trois sites concurrents dans le secteur de la pâtisserie haut de gamme : Angelina Paris, La Maison du Chocolat, et Pierre Hermé.

Ce que j'en ai retenu, c'est trois choses essentielles. D'abord, la mise en avant visuelle des produits avec de grandes photos — dans ce secteur, l'image vend avant le texte. Ensuite, une navigation claire par catégorie — l'utilisateur doit trouver ce qu'il cherche en deux clics maximum. Et enfin, une identité visuelle forte et cohérente qui donne immédiatement envie et distingue la marque.

Ce sont ces trois principes que j'ai appliqués à SamyDessert : grandes photos, navigation simple, et une palette de couleurs évocatrice.

Slide 5 — Personas ⏱ 1min15

Pour cadrer les besoins utilisateurs et éviter de développer des fonctionnalités inutiles, j'ai défini deux personas.

Marie, 34 ans, est une cliente régulière. Elle commande depuis son téléphone, elle veut quelque chose de simple et rapide. Elle n'a pas envie de créer un compte compliqué ni de naviguer dans dix menus. Ce persona m'a poussé vers un design mobile-first et un tunnel de commande le plus court possible.

Thomas, 28 ans, est passionné de cuisine. Il cherche des recettes accessibles, avec des ingrédients courants, qu'il peut reproduire chez lui. Mais parfois, il préfère commander directement. Ce persona justifie le concept hybride du site — recettes gratuites ET boutique.

Ces deux profils ont orienté les priorités fonctionnelles dès le début du projet.

Slide 6 — User Flow ⏱ 1min

Avant de concevoir les maquettes, j'ai modélisé le parcours utilisateur principal. Le voici : l'utilisateur arrive sur l'accueil, consulte le catalogue, ouvre une fiche produit, ajoute au panier, passe commande, effectue le paiement via Stripe, et reçoit un email de confirmation.

Ce schéma m'a servi de fil conducteur tout au long du développement. À chaque fonctionnalité, je me demandais : est-ce que ça s'intègre naturellement dans ce parcours ? Est-ce que ça le simplifie ou est-ce que ça le complique ? C'est aussi ce parcours que je vais vous montrer lors de la démonstration.

Slide 7 — Wireframe ⏱ 1min

Avant de coder, j'ai réalisé des wireframes basse fidélité sur Figma. L'objectif, c'est de valider la structure des pages — la disposition des éléments, la navigation, la hiérarchie de l'information — avant de s'engager dans le développement.

L'intérêt des wireframes, c'est que modifier une boîte rectangulaire sur Figma prend 10 secondes. Modifier un composant déjà développé peut prendre 30 minutes. En validant la structure tôt, on évite de refaire du travail en cours de route. C'est une étape que j'avais tendance à vouloir sauter pour aller directement au code, mais j'ai compris que ça coûte en fait beaucoup plus de temps de ne pas le faire.

Slide 8 — Identité visuelle & Design system ⏱ 1min30

Le logo est un "S" stylisé que j'ai conçu avec Affinity Designer. Pour la palette, j'ai choisi quatre couleurs qui évoquent directement la pâtisserie : le chocolat foncé pour les textes et les fonds, la framboise pour les boutons d'action et les accents, la pistache pour les badges de succès, et le crème pour le fond principal. Ensemble elles donnent immédiatement une identité "artisanal" et "gourmand".

Ce qui est important ici, c'est que ces couleurs ne sont pas juste des choix esthétiques — elles sont déclarées comme des **tokens CSS** dans `app.css` avec des noms sémantiques. Par exemple `--color-primary` pour le chocolat, `--color-accent` pour la framboise. Si demain le client veut changer la couleur principale, je modifie une seule ligne et tout le site se met à jour.

Le design system va plus loin : j'ai des composants réutilisables — des boutons, des cards produit, un header. Chaque composant a ses styles définis une seule fois et s'utilise partout de manière cohérente. J'expliquerai comment ils sont organisés en détail dans la slide sur la structure du projet.

Slide 9 — Responsive Design & Accessibilité ⏱ 1min30

Le site est développé en **mobile-first** avec Tailwind CSS v4. L'approche mobile-first signifie qu'on part du plus petit écran et on ajoute des styles pour les écrans plus grands — et non l'inverse. Ça force à prioriser le contenu essentiel. J'ai testé trois breakpoints : mobile, tablette et desktop.

Pour l'accessibilité, j'ai appliqué plusieurs principes. Les contrastes de couleurs ont été vérifiés avec l'outil Colorable pour être conformes WCAG 2.1 niveau AA. J'ai utilisé la police Luciole, qui est spécialement conçue pour les personnes dyslexiques — elle a une reconnaissance accrue des lettres qui se ressemblent. Tous les éléments interactifs ont un focus visible pour la navigation au clavier. Et j'ai utilisé des attributs ARIA sur les éléments dynamiques — notamment sur le bouton panier qui annonce le nombre d'articles aux lecteurs d'écran.

L'accessibilité n'était pas une option, c'était une contrainte du projet dès le départ, et ça m'a vraiment changé la façon de penser le HTML.

Slide 10 — Stack technique ⏱ 1min30

Backend — PHP 8.3 + Symfony 7.4. Symfony est organisé autour d'un kernel HTTP et d'un container d'injection de dépendances. Chaque service est instancié une seule fois et injecté automatiquement par type-hint dans les constructeurs — c'est l'autowiring. J'utilise PHP 8.3 pour les enums natifs et les readonly properties.

Frontend — Twig + Tailwind CSS v4 + Stimulus. Twig échappe automatiquement toutes les variables — protection XSS par défaut. Tailwind v4 abandonne le fichier `tailwind.config.js` : les tokens sont déclarés directement en CSS avec la directive `@theme` et deviennent des custom properties natives `--color-accent`, `--color-primary`. Elles sont utilisables partout en CSS sans plugin.

Stimulus ne manipule pas de DOM virtuel — il connecte des attributs HTML (`data-controller`, `data-action`, `data-target`) à des classes JavaScript. Aucune compilation, aucun bundle nécessaire.

Temps réel — Symfony UX Live Components + Turbo. Un Live Component est un composant Twig qui se re-rend côté serveur via une requête AJAX transparente quand son état change. Turbo Frames fait pareil pour des portions de page entières — seul le fragment ciblé est remplacé, pas toute la page.

Doctrine ORM. Doctrine utilise le pattern **Data Mapper** — les entités ne savent pas qu'elles sont persistées. C'est l'`EntityManager` qui les observe via l'Unit of Work et génère les requêtes SQL au `flush()`.

AssetMapper. Pas de Node.js, pas de `npm install`, pas d'étape de build. AssetMapper expose les fichiers JS directement au navigateur avec un `importmap` — le navigateur résout les modules ES nativement. Webpack Encore n'est plus nécessaire.

Slide 11 — Architecture MVC ⏱ 1min30

Je vais vous expliquer le MVC en suivant une vraie requête du projet : `POST /panier/ajouter/5`.

Le navigateur envoie cette requête. Le **kernel HTTP** de Symfony la reçoit et déclenche une chaîne d'événements. Le **routeur** compare l'URL et la méthode aux attributs `#[Route]` de tous les controllers — il trouve `#[Route('/panier/ajouter/{id}', methods: ['POST'])]` dans `PanierController` et extrait `id = 5`.

Symfony instancie `PanierController` et injecte `PanierService` via le container d'injection de dépendances — c'est l'autowiring par type-hint. La méthode `ajouter(int $id, PanierService $panier)` est appelée avec `$id = 5`.

Le **Controller** ne fait rien de plus que déléguer : `$panier->ajouter(5)`. Toute la logique — lire la session, incrémenter la quantité, sauvegarder — est dans le **Service**. Le Controller ajoute un message flash et retourne une `RedirectResponse` vers `/panier`.

Aucune entité Doctrine n'est touchée ici — le panier est en session. Si c'était une commande, le Service appellerait le **Repository** qui génère la requête SQL via l'EntityManager. Le Controller ne touche jamais la base directement.

Ce découpage rend chaque couche testable indépendamment : `PanierServiceTest` teste la logique sans simuler de requête HTTP.

Slide 12 — Structure du projet ⏱ 1min30

Pour les composants frontend, j'ai appliqué la méthode **Atomic Design** via les Symfony UX Twig Components. L'idée, c'est d'organiser les composants du plus simple au plus complexe.

Les atomes sont les éléments de base qui ne peuvent pas être décomposés davantage : un bouton, un badge, un input, un lien. Les molécules combinent ces atomes pour créer un élément fonctionnel : une carte produit, un bouton panier, la notation en étoiles. Les organismes assemblent les molécules pour former des sections entières de l'interface : le header, le footer, la grille de produits, le panier live.

Cette approche me permet de modifier un atome une seule fois et de voir le changement répercuté partout où il est utilisé.

Pour les interactions JavaScript, j'ai 14 controllers Stimulus. Chacun a une responsabilité unique et clairement définie. Par exemple `carousel` gère uniquement le carousel de la page d'accueil, `cart-sidebar` gère uniquement l'ouverture et la fermeture du panier, `favori` gère le toggle favori avec l'appel AJAX. Je n'ai pas de fichier JavaScript monolithique — chaque comportement est encapsulé dans son propre fichier.

Slide 13 — Code : Atomic Design — Atom & Molécule ⏱ 1min30

Je vais vous montrer concrètement ce que signifie Atomic Design dans le code.

L'atome `Button.html.twig` définit les variants avec les tokens CSS du design system. `primary` utilise `bg-accent` — c'est la couleur framboise définie en slide 8. Si je change la framboise dans `app.css`, tous les boutons primary se mettent à jour automatiquement. Le composant s'utilise en une ligne : `<twig:Atoms:Button variant="ghost">Annuler</twig:Atoms:Button>`.

La molécule `InputField.html.twig` assemble plusieurs atomes — un `Label`, un `Input` — et ajoute de la logique que chaque atome seul ne peut pas avoir : la gestion des erreurs avec `role="alert"` pour les lecteurs d'écran, et l'attribut `aria-invalid` pour indiquer un champ invalide. La molécule s'utilise aussi en une ligne, mais elle encapsule tout ce travail invisible.

C'est ça la vraie valeur d'Atomic Design : les atomes sont simples et réutilisables, les molécules ajoutent de la sémantique et de l'accessibilité.

Slide 14 — Code : Controller Stimulus — Carousel ⏱ 2min

Le carousel est la réalisation frontend dont je suis le plus fier — c'est une classe JavaScript vanilla conçue à partir d'un tutoriel Grafikart et étendue avec plusieurs mécanismes.

Premier point — mode infini par clonage.

Je reconstruit le tableau `this.items` avec le spread operator : je prends les N derniers nœuds du DOM avec `slice(-offset)`, je les clone avec `cloneNode(true)` — clone profond, avec tout le HTML interne — et je les prépose. Idem pour les N premiers que je place à la fin. Le DOM contient donc plus de slides que la réalité.

Quand l'utilisateur arrive sur un clone, `onTransitionEnd` se déclenche — c'est un listener sur l'événement `transitionend` du navigateur. Je désactive la transition CSS avec `setTransition(false)`, ce qui revient à passer `transition-duration` à `0ms` — le mouvement devient instantané. Je recalcule `currentItem` avec la formule `+= items.length - 2 * offset` pour sauter sur le vrai élément correspondant, j'appelle `translate()` qui met à jour le `transform: translateX(...)` du conteneur, puis je réactive la transition. L'utilisateur ne voit rien.

Deuxième point — protection contre le double-clic.

`isAnimating` est un booléen d'instance. Quand `next()` est appelé, si `isAnimating` est déjà `true` on sort immédiatement avec `return`. On le repasse à `false` dans un `setTimeout` calé sur la durée de la transition CSS. J'aurais pu utiliser `transitionend`, mais `setTimeout` est plus robuste quand la transition est interrompue.

Troisième point — accessibilité avec `inert`.

`item.inert = !isActive` écrit directement la propriété booléenne native `HTMLElement.inert` sur le nœud DOM. Un seul attribut désactive simultanément le focus clavier, les événements pointer, et masque l'élément des lecteurs d'écran. `aria-hidden` seul ne bloque pas le focus — `inert` est la seule solution propre. Seule la slide visible reste interactive.

Slide 15 — Base de données ⏱ 1min30

La base de données compte 6 entités et une table de liaison enrichie. Je veux d'abord expliquer les choix d'architecture, puis trois décisions techniques.

Architecture des relations.

Le panier n'est pas une entité en base de données — il est stocké en session PHP via `PanierService`. Un tableau `[produitId => quantite]` en session, c'est suffisant pour ce besoin et ça évite des écritures en base à chaque ajout.

Les favoris, c'est le contraire : ils doivent persister entre les sessions. J'aurais pu créer une entité `Favori`. J'ai préféré une relation `ManyToMany` directement sur `Utilisateur` — Doctrine génère automatiquement les tables de jointure `utilisateur_produit_favori` et `utilisateur_recette_favori`. Pas de colonne supplémentaire nécessaire sur la relation, donc `ManyToMany` automatique suffit.

`CommandeProduit` en revanche est une entité intermédiaire explicite — ce qu'on appelle un **ManyToMany avec payload**. Une commande contient plusieurs produits et j'ai besoin de stocker des données sur la relation elle-même : la quantité commandée et le prix unitaire au moment de l'achat. Doctrine ne peut pas stocker ces colonnes sur une jointure automatique. J'ai donc créé l'entité `CommandeProduit` avec ses propres champs `quantite` et `prixUnitaire`.

Trois décisions techniques intentionnelles.

Première : les prix sont en `DECIMAL(8,2)` et non en `float`. Les floats ont des erreurs d'arrondi en base binaire — `0.1 + 0.2` ne vaut pas exactement `0.3`. Sur des montants financiers, c'est inacceptable.

Deuxième : le `prixUnitaire` est dupliqué dans `CommandeProduit` au moment de la commande. Si je modifie le prix d'un produit demain, les anciennes commandes conservent le prix exact auquel elles ont été passées. Sans cette duplication, l'historique comptable serait faussé.

Troisième : j'utilise des enums PHP natifs pour les statuts — `EnAttente`, `Confirmee`, `Annulee`. Impossible de stocker une valeur hors de cet ensemble, l'IDE autocompète, et Doctrine les sérialise proprement en base. C'est PHP 8.1, disponible parce qu'on est sur PHP 8.3.

Slide 16 — Fonctionnalités : Catalogue & Recettes ⏱ 1min

Le catalogue permet de filtrer par catégorie, trier par prix ou par note, et de faire une recherche par mot-clé. Ce qui est intéressant techniquement, c'est que les filtres et le tri utilisent des **Turbo Frames** : quand on change un filtre, seule la grille de produits se met à jour sur la page, pas le header ni le footer. Ça donne une navigation fluide, sans rechargement visible, sans écrire de JavaScript custom. Turbo gère ça automatiquement.

Sur les fiches produits, on peut zoomer sur l'image au clic. Le zoom est géré par le controller Stimulus `image-zoom`. On peut aussi ajouter le produit aux favoris directement depuis la grille, via une requête AJAX.

La section recettes utilise le même organism que le catalogue — même composant, même expérience utilisateur.

Slide 17 — Fonctionnalités : Panier ⏱ 1min

Le panier est une sidebar persistante qui s'ouvre en superposition sur la page. Elle s'ouvre et se ferme via le controller Stimulus `cart-sidebar`, sans rechargement.

Le contenu se met à jour en temps réel grâce aux **Live Components**. Quand j'ajoute un produit depuis n'importe quelle page, le compteur dans le header et le contenu de la sidebar reflètent immédiatement le changement — sans JavaScript custom, sans écrire un seul appel fetch. Le Live Component envoie automatiquement une requête au serveur et re-rend le composant.

Pour vider le panier, une boîte de confirmation s'affiche avant. J'utilise l'élément `<dialog>` natif HTML5 — pas de librairie externe, pas de code custom. C'est géré par le controller Stimulus `confirm`.

Slide 18 — Fonctionnalités : Commande & Paiement ⏱ 1min15

Le tunnel de commande se déroule en quatre étapes. L'utilisateur confirme le récapitulatif de son panier, saisit son adresse de livraison, est redirigé vers Stripe Checkout pour le paiement, puis arrive sur une page de confirmation.

Ce que je veux insister ici, c'est que je ne touche jamais aux données de carte bancaire. L'utilisateur saisit sa carte directement sur les serveurs de Stripe — mon serveur ne voit jamais ces informations. Une fois le paiement validé, Stripe envoie un événement à mon webhook.

Le webhook, c'est une route sur mon serveur que Stripe appelle en coulisse, serveur à serveur. C'est plus fiable que de faire confiance au retour navigateur — si l'utilisateur ferme la fenêtre après avoir payé, le webhook a quand même été reçu. J'en parlerai en détail avec le code.

Slide 19 — Emails transactionnels ⏱ 45s

Quatre emails sont envoyés automatiquement. À l'inscription, l'utilisateur reçoit un lien de vérification — son compte n'est pas actif tant qu'il n'a pas cliqué. Quand son compte est vérifié, il reçoit un email de bienvenue. Quand une commande est confirmée, il reçoit une confirmation avec la facture PDF en pièce jointe. Et si une commande est annulée, il reçoit une notification.

Ces emails sont envoyés via Resend API. J'avais d'abord configuré un serveur SMTP classique, mais il était bloqué par Railway. Resend utilise le protocole HTTP — ça contourne le problème. Le domaine `samydessert.fr` a été acheté séparément et vérifié dans Resend via des enregistrements DNS.

Slide 20 — Fonctionnalités : Administration ⏱ 1min

L'interface d'administration est réalisée avec EasyAdmin. Elle donne accès à la gestion complète des produits avec upload d'images, des recettes, des commandes avec modification de statut, des utilisateurs et des avis clients.

Il y a aussi un tableau de bord avec des statistiques en temps réel : nombre de produits, commandes, utilisateurs, et aperçu des dernières commandes.

L'upload d'images est géré par VichUploader — les images sont stockées localement sur le serveur, pas sur un service tiers. L'accès à l'admin est protégé : seuls les comptes avec le rôle `ROLE_ADMIN` peuvent y accéder.

Slide 21 — Soins UX ⏱ 1min

Deux soins UX que je voulais mettre en avant.

Les messages d'erreur de connexion sont volontairement vagues : "adresse email ou mot de passe incorrect". Si on disait "email introuvable", un attaquant pourrait tester des emails pour savoir quels comptes existent. C'est une recommandation directe de l'OWASP — c'est un choix délibéré, pas un oubli.

Le deuxième soin, c'est le bouton panier. C'est un **Live Component** Symfony UX. La classe PHP `BoutonPanier` est annotée `#[AsLiveComponent]`. La propriété `$produitId` est annotée `#[LiveProp]` — c'est ce qui permet à Twig de la connaître. La méthode `ajouter()` est annotée `#[LiveAction]` — quand le bouton est cliqué, Symfony UX envoie une requête AJAX vers cette méthode, re-rend le composant, et émet l'événement `panierUpdated` pour mettre à jour le compteur dans le header.

Le pattern `data-loading` dans le template Twig fait le reste : pendant l'action, l'icône se masque (`data-loading="hide"`) et le spinner apparaît (`data-loading="show"`). L'utilisateur voit que son action est en cours — sans écrire une seule ligne de JavaScript custom.

Slide 22 — Sécurité ⏱ 1min30

Six points de sécurité principaux.

Les tokens **CSRF** protègent tous les formulaires. Un token unique est généré pour chaque session et vérifié à la soumission — ça empêche un site malveillant de soumettre un formulaire à la place de l'utilisateur.

Twig échappe automatiquement toutes les variables affichées dans les templates. Impossible d'injecter du HTML ou du JavaScript via les données.

Doctrine utilise des requêtes préparées — les paramètres ne sont jamais concaténés dans les requêtes SQL, ce qui rend l'injection SQL impossible.

L'accès aux routes est contrôlé par les rôles Symfony. Les pages de l'espace client nécessitent `ROLE_USER`, l'admin nécessite `ROLE_ADMIN`.

Les mots de passe sont hachés avec l'algorithme `auto` de Symfony — il choisit le meilleur algorithme disponible selon la version PHP. Sur PHP 8.3, c'est Sodium. Les mots de passe en clair ne sont jamais stockés.

Et les données de paiement ne transitent jamais par mon serveur — Stripe gère ça de son côté.

Slide 23 — Code : Webhook Stripe & idempotence ⏱ 2min

Je vais vous montrer un exemple concret de code. C'est le webhook Stripe, que je considère comme la fonctionnalité la plus intéressante techniquement du projet.

Le problème de départ : Stripe peut envoyer le même événement plusieurs fois. Si leur serveur ne reçoit pas de réponse `200 OK` dans les délais — par exemple parce que mon serveur était lent — il réessaie. Sans

protection, la commande serait confirmée deux fois et l'utilisateur recevrait deux emails.

La solution en deux parties.

D'abord, je vérifie la **signature** de la requête. Stripe signe chaque requête avec une clé secrète partagée — un algorithme HMAC. La méthode `Webhook::constructEvent()` fait cette vérification. Si la signature ne correspond pas, je rejette la requête immédiatement avec un HTTP 400. Ça garantit que seul Stripe peut déclencher ce webhook — personne ne peut l'appeler depuis l'extérieur.

Ensuite, avant de confirmer la commande, je vérifie son statut actuel. Si c'est déjà "confirmée", la condition est fausse — on ne fait rien, on renvoie quand même `200 OK` pour dire à Stripe qu'on a bien reçu l'événement. Si c'est "en attente", on confirme, on flush en base, on envoie l'email.

C'est le principe d'**idempotence** : exécuter l'opération plusieurs fois donne exactement le même résultat qu'une seule fois. C'est un concept fondamental dans les architectures qui traitent des événements asynchrones ou des paiements.

Slide 24 — Code : Favoris AJAX — Stimulus ↔ PHP ⏱ 2min

Deuxième exemple de code : le système de favoris. L'objectif est d'ajouter ou retirer un favori sans rechargement de page, avec une mise à jour visuelle immédiate de l'icône.

Côté PHP, j'ai une seule route POST qui gère les deux cas — ajout et retrait — avec la même URL. Le principe est simple : si l'entité est déjà dans les favoris de l'utilisateur, on la retire. Sinon, on l'ajoute. On `flush()` en base, et on retourne un JSON avec juste `{ favori: true }` ou `{ favori: false }`. Le Controller ne fait que ça. La relation many-to-many est gérée par Doctrine.

Côté JavaScript, le controller Stimulus fait une requête `fetch` vers cette route. Deux points intéressants.

Le premier : si le serveur répond `401`, l'utilisateur n'est pas connecté. Au lieu d'afficher une alerte globale, je cherche le controller `flash-tooltip` sur la même carte produit et je lui demande de s'afficher — "connectez-vous pour ajouter aux favoris". C'est ce qu'on appelle la communication entre controllers Stimulus.

Le deuxième : quand la réponse arrive, je fais juste `this.activeValue = data.favori`. Et Stimulus appelle automatiquement la méthode `activeValueChanged()` — je n'ai pas à l'appeler moi-même. Cette méthode met à jour l'animation de l'icône et l'attribut `aria-label` pour l'accessibilité. Le JavaScript ne contient aucune logique métier — uniquement de la mise à jour d'interface.

Slide 25 — Code : PanierService — Service & accès aux données ⏱ 1min30

Troisième exemple de code, qui montre une couche différente : le pattern Service et l'accès à la base de données avec Doctrine.

Le principe du pattern Service, c'est que le Controller ne contient aucune logique métier — il délègue tout. Tout ce qui concerne le panier est dans `PanierService`. Si je veux tester le panier, je teste le Service directement, sans simuler de requête HTTP. C'est exactement ce que fait `PanierServiceTest`.

La méthode `getLignes` montre un point technique important. Le panier en session, c'est juste un tableau d'identifiants et de quantités. Pour récupérer les vrais objets `Produit`, j'utilise `findBy(['id' => array_keys($panier)])`. Ça fait **une seule requête SQL** pour tous les produits du panier, quelle que soit la quantité d'articles. Si j'avais fait un `findById` dans une boucle, j'aurais N requêtes. C'est le problème classique du N+1 que j'ai évité intentionnellement.

La méthode `ajouter` est volontairement minimaliste : trois lignes. Elle lit la session, incrémente ou initialise la quantité, et sauvegarde. Pas de base de données, pas de HTTP — c'est ça un Service bien découpé.

Slide 26 — Tests ⏱ 1min30

J'ai mis en place une suite de 74 tests automatisés avec PHPUnit 12 — 26 unitaires et 48 fonctionnels. Les tests unitaires vérifient la logique métier isolée : calcul du panier, total d'une commande, blocage de connexion pour un compte non vérifié. Les tests fonctionnels simulent de vraies requêtes HTTP vers le kernel Symfony.

Chaque test s'exécute sur une base isolée `samydessert_test`, réinitialisée entre chaque test via Zenstruck Foundry.

Je veux montrer un exemple concret de test que j'ai dû corriger. Le test du formulaire de contact envoyait un POST sans token CSRF — le contrôleur rejetait la requête et redirigeait vers la page de connexion au lieu de `/contact`. Le test échouait. La correction : faire d'abord un GET pour récupérer le vrai token CSRF depuis le formulaire, puis le soumettre avec le POST. C'est exactement ce que fait un vrai navigateur — et le test le reflète maintenant fidèlement.

Slide 27 — Déploiement ⏱ 1min

En développement, j'utilise Docker Compose avec 6 services : nginx comme serveur web, php-fpm pour exécuter PHP, mysql pour la base de données, adminer pour visualiser la base via un navigateur, un container d'initialisation qui crée la base et joue les migrations au démarrage, et un container pour la compilation des assets. N'importe qui peut cloner le projet et lancer `docker compose up` — aucune installation locale requise sur la machine.

En production, le site tourne sur Railway. Le déploiement est automatique : chaque push sur la branche `main` déclenche un build et un déploiement du container Docker. Les variables sensibles — clés Stripe, clé Resend, mot de passe de base de données — sont stockées dans Railway, jamais dans le code ni dans le repository Git.

Slide 28 — Gestion de version avec Git ⏱ 45s

Je travaille avec deux branches. L'idée est d'isoler le développement en cours sur `dev` pour ne jamais pousser directement du code instable en production. Sur ce projet solo j'ai parfois travaillé directement sur `main`, mais le principe est là : `dev` pour les changements en cours, `main` pour ce qui est stable et prêt à déployer. C'est une pratique standard dans les projets professionnels où plusieurs personnes travaillent en parallèle.

Je suis la convention **Conventional Commits** : `feat:` pour une nouvelle fonctionnalité, `fix:` pour une correction de bug, `refactor:` pour du code restructuré sans changer le comportement, `docs:` pour la

documentation. Ça permet de lire l'historique git et de comprendre immédiatement la nature de chaque commit sans ouvrir le diff.

Slide 29 — Veille technologique ⏱ 1min

Ma veille repose sur plusieurs sources complémentaires. Stack Overflow et GitHub Issues pour les problèmes concrets — quand je bloque sur un bug, c'est souvent là que quelqu'un a déjà eu le même problème. YouTube pour les démonstrations visuelles, notamment Grafikart qui traite beaucoup de PHP et Symfony. Et les outils d'IA pour comprendre rapidement un concept ou explorer des approches. Mais je vérifie toujours les réponses — l'IA peut se tromper, surtout sur des versions récentes de frameworks.

Sur ce projet, j'ai adopté plusieurs technologies récentes que je n'avais pas utilisées avant. Tailwind CSS v4 sans fichier de configuration JavaScript. PHP 8.3 avec les enums natifs — ça remplace des constantes de classe et garantit des valeurs valides. Resend API à la place de SMTP, plus fiable en production. Et AssetMapper qui remplace Webpack Encore — pour ce projet, il n'y a plus besoin de `node_modules` ni de build step.

Slide 30 — Difficultés rencontrées ⏱ 1min15

Cinq difficultés majeures que j'ai rencontrées et résolues.

Le **webhook Stripe** ne pouvait pas recevoir de requêtes en local — mon serveur de dev n'est pas accessible depuis Internet. J'ai utilisé Stripe CLI avec la commande `stripe listen` qui crée un tunnel et relaie les événements Stripe vers localhost.

Les **emails SMTP** étaient bloqués depuis Railway — la plateforme bloque les connexions sortantes sur les ports email pour des raisons de sécurité réseau. J'ai migré vers Resend API qui utilise HTTPS — ça contourne complètement le problème.

Le **watcher Tailwind** sur Docker Windows avait un problème avec les chemins de fichiers — il ne détectait pas les changements à cause du système de fichiers virtualisé. J'ai mis en place un script de recompilation toutes les 3 secondes comme contournement.

Les **Turbo Frames** nécessitent d'identifier précisément quelle portion de la page doit être mise à jour. Au départ j'avais mal délimité les frames, ce qui causait des rechargements complets. Il a fallu déboguer en inspectant les requêtes réseau.

VichUploader ne trouvait pas les chemins d'images générés par AssetMapper — les deux systèmes avaient des conventions différentes. J'ai dû configurer manuellement le mapping entre les deux.

Slide 31 — Améliorations futures ⏱ 45s

Quelques pistes d'amélioration. Fonctionnellement, j'aimerais ajouter la liste des ingrédients dans les recettes pour que l'utilisateur puisse acheter directement ce dont il a besoin. Une modération des avis avant publication. Des variations de produits — tailles ou parfums différents.

Techniquement, les priorités sont : un token CSRF sur les actions AJAX comme les favoris — c'est une limite que j'ai identifiée moi-même. Un token de vérification email avec expiration — actuellement les liens de

vérification n'expirent pas. Et l'internationalisation avec `symfony/translation` pour préparer une version multilingue.

Slide 32 — Démonstration live ⏱ 5min

Je vais maintenant vous montrer le site en production.

Script de démo à suivre :

1. **Accueil** — montrer le carousel, les produits mis en avant, le header responsive
2. **Catalogue** — changer de catégorie (montrer que seule la grille se recharge — Turbo Frames), zoomer sur une image, cliquer sur le favori d'un produit non connecté (tooltip "connectez-vous")
3. **Connexion** — se connecter avec `test@test.com` / `password123`
4. **Catalogue connecté** — ajouter un favori (voir l'icône se remplir), ajouter un produit au panier (voir l'animation du bouton et le compteur du header)
5. **Panier** — ouvrir la sidebar, modifier une quantité, montrer la mise à jour en temps réel
6. **Commande** — cliquer sur commander, saisir une adresse, voir le récapitulatif
7. **Paiement Stripe** — carte `4242 4242 4242 4242`, date future, CVC quelconque, cliquer "Payer"
8. **Confirmation** — montrer la page de confirmation, panier vidé
9. **Admin** — se déconnecter, se reconnecter avec `samy@admin.com` / `password123`, montrer le tableau de bord, la liste des produits, l'édition d'un produit

> **Si le site est indisponible** : ouvrir les captures d'écran dans `Soutenance/DossierProjet/captures/`

Slide 33 — Bilan personnel ⏱ 1min30

Ce projet m'a appris énormément de choses. Structurer un projet Symfony de A à Z — du Dockerfile jusqu'aux tests. Travailler dans un environnement Docker sans installation locale. Intégrer des services tiers comme Stripe pour le paiement, Resend pour les emails, VichUploader pour les images.

Mais ce qui m'a le plus apporté, c'est comprendre comment toutes les briques s'assemblent. Du token CSS dans `app.css` jusqu'au webhook qui arrive depuis les serveurs de Stripe, en passant par les entités Doctrine, les templates Twig et les controllers Stimulus. Voir chaque couche communiquer avec la suivante, c'est ce qui donne une vraie compréhension du développement web.

L'accessibilité m'a aussi surpris — ça change profondément la façon de penser le HTML. Ce n'est pas juste ajouter un attribut `alt` sur les images, c'est repenser la structure, les attributs ARIA, la navigation clavier dès la conception.

Ce que je ferais différemment : j'écrirais les tests en parallèle du développement, pas après. Et je passerais plus de temps sur la modélisation de la base de données avant de commencer à coder — j'ai dû faire des migrations correctives en cours de route, ce qui aurait pu être évité.

Slide 34 — Questions

Merci pour votre attention. Je suis prêt pour vos questions.

Prépare ces questions — réponses courtes et directes

Pourquoi Symfony et pas Laravel ? Symfony est plus utilisé en entreprise en France, plus modulaire, et c'est le framework que j'ai appris en formation. Laravel est plus rapide à démarrer, mais Symfony donne plus de contrôle sur l'architecture. Les deux sont bons — c'est surtout une question de contexte.

Pourquoi Railway et pas un VPS ? Railway simplifie le déploiement avec un Dockerfile existant et s'intègre directement avec Git. Un VPS demanderait de configurer nginx, SSL, les sauvegardes, la supervision — c'est pertinent en production réelle, mais hors scope pour ce projet de formation.

C'est quoi un Live Component concrètement ? C'est un composant Twig qui se re-rend côté serveur quand son état change, sans écrire de JavaScript manuellement. Le panier est un Live Component : quand j'ajoute un produit, le composant fait une requête AJAX transparente au serveur, se re-rend, et remplace son propre HTML dans la page. Je n'écris pas de fetch, pas de DOM manipulation.

Comment fonctionne l'idempotence du webhook ? Stripe peut envoyer le même événement plusieurs fois en cas de timeout. Avant de confirmer la commande, je vérifie que son statut est encore "en attente". Si c'est déjà "confirmée", je ne fais rien — je renvoie juste un 200 OK pour dire à Stripe que j'ai bien reçu. Ça évite les doubles confirmations.

Tu aurais fait quoi différemment ? Les tests dès le début, en parallèle du développement. Et une modélisation plus précise de la base de données avant de coder — j'ai dû faire des migrations correctives.

Le RGPD dans ton projet ? Je collecte les données nécessaires au compte et aux commandes. Les mots de passe sont hachés, les données de paiement ne transitent jamais par mon serveur. C'est un vrai sujet d'amélioration — il manque une politique de confidentialité complète et une bannière cookies conforme.

C'est quoi Atomic Design ? C'est une méthode de structuration des composants UI du plus simple au plus complexe. Un atome c'est un bouton seul. Une molécule c'est une carte produit qui assemble un bouton, une image et un badge. Un organisme c'est la grille produits qui assemble plusieurs cartes. L'intérêt : modifier un atome répercute le changement partout où il est utilisé.

Pourquoi dupliquer le prix dans CommandeProduit ? Si je ne le duplique pas et que je change le prix d'un produit demain, toutes les commandes passées afficheraient le nouveau prix. L'historique serait faux. En dupliquant le prix au moment de la commande, chaque commande garde le prix exact auquel l'achat a été fait.

Comment tu as testé le webhook en local ? Avec Stripe CLI, commande `stripe listen --forward-to localhost:8000/webhook/stripe`. Ça crée un tunnel entre les serveurs de Stripe et mon localhost, et relaie tous les événements en temps réel. Je peux aussi déclencher des événements manuellement avec `stripe trigger checkout.session.completed`.